



SDU International Summer School

Introduction to R

Day 5 - Data wrangling with Tidyverse

Paola Vazquez Castillo

07 August 2026

Contents

1	Introduction	2
2	Importing the data	2
3	Tidyverse and tibbles	3
4	Data Transformation	4
4.1	Missing values, handled systematically	4
4.2	Choosing columns: <code>select()</code>	5
4.3	Choosing rows: <code>filter()</code> and <code>arrange()</code>	6
4.4	Creating new variables: <code>mutate()</code>	9
4.5	Recoding and categorising: <code>case_when()</code> and <code>recode()</code>	10
4.6	Summarizing	11
5	Pipes	11
5.1	<code>group_by()</code>	11
6	Reshaping data	12
6.1	Converting to long	13
6.2	Converting to wide	14
7	Joining two datasets	14
8	Bringing everything together	15
9	Tidyverse and R base programming	16
10	Glossary	16
11	Exercises	17

1 Introduction

Yesterday we learned about packages and test their strengths. We also discussed about data analysis and ended the session with a first taste of three tidyverse verbs: `select()`, `filter()`, and `mutate()`. Today we extend the data wrangling process by adding more tidying and transformation functions into our tool box.

2 Importing the data

When we work with data, we usually have a question in mind. That is the entire motivation for the analysis. Our question for the day is: does sleep matter for grades? We will use a dataset that paired Fitbit sleep tracking with grades for 634 first-year students (it mirrors the data released with Creswell and colleagues, 2023, in PNAS).

Unlike Day 4, today the data lives in files on your machine, so we start by importing them. Set your working directory to the folder that holds the two files, then read them in.

```
# setwd("path/to/your/folder") # point R at the folder with the csv files
```

```
sleep <- read.csv("student_sleep.csv")
studies <- read.csv("studies.csv")
```

```
head(sleep)
```

```
##  student_id study gender first_generation underrepresented avg_sleep_hours
## 1      293    1 Female                0                0          6.17
## 2      294    1 Female                0                0          6.78
## 3      297    1 Female                0                1          4.23
## 4      303    1 Female                0                0          6.86
## 5      304    1 Female                0                1          6.80
## 6      305    1  Male                1                1          5.78
##  daytime_sleep_minutes sleep_midpoint_minutes bedtime_variability
## 1              11.7              500.3              0.0500
## 2              49.7              359.5              0.0476
## 3             178.5              398.2              0.1639
## 4              26.1              532.1              0.7854
## 5              71.6              354.0              0.1538
## 6              78.1              541.3              1.0689
##  nights_tracked_fraction prior_gpa term_gpa term_units term_load_z
## 1              0.931        4.00    3.76        49        0.112
## 2              0.862        4.00    3.55        49        0.112
## 3              0.586        3.00    3.59        46       -0.604
## 4              0.931        3.80    3.46        39       -2.275
## 5              0.931        3.17    3.50        39       -2.275
## 6              0.724        2.37    2.54        49        0.112
```

```
#head(studies)
```

The main file `student_sleep.csv` has one row per student. The smaller `studies.csv` describes each of the five study samples (which university it came from, which semester, how many students).

A quick look at the structure, as in Day 2:

```
str(sleep)
```

```
## 'data.frame': 634 obs. of 14 variables:
## $ student_id : int 293 294 297 303 304 305 306 308 314 315 ...
## $ study : int 1 1 1 1 1 1 1 1 1 1 ...
## $ gender : chr "Female" "Female" "Female" "Female" ...
## $ first_generation : int 0 0 0 0 0 1 0 0 0 0 ...
## $ underrepresented : int 0 0 1 0 1 1 0 1 0 0 ...
## $ avg_sleep_hours : num 6.17 6.78 4.23 6.86 6.8 5.78 5.67 6.91 6.35 7.17 ...
## $ daytime_sleep_minutes : num 11.7 49.7 178.5 26.1 71.6 ...
## $ sleep_midpoint_minutes : num 500 360 398 532 354 ...
## $ bedtime_variability : num 0.05 0.0476 0.1639 0.7854 0.1538 ...
## $ nights_tracked_fraction: num 0.931 0.862 0.586 0.931 0.931 0.724 0.759 0.931 0.966 0.
## $ prior_gpa : num 4 4 3 3.8 3.17 2.37 3.4 2.53 2.79 4 ...
## $ term_gpa : num 3.76 3.55 3.59 3.46 3.5 2.54 3.5 2.4 3.37 3.61 ...
## $ term_units : int 49 49 46 39 39 49 45 48 49 47 ...
## $ term_load_z : num 0.112 0.112 -0.604 -2.275 -2.275 ...
```

```
dim(sleep)
```

```
## [1] 634 14
```

3 Tidyverse and tibbles

Today we will see how to tidy up a dataset using the package `tidyverse`. So the first step is to load the package.

```
# install.packages("tidyverse")
```

```
# Load the package in R
```

```
library(tidyverse)
```

The purpose of the tidyverse is to work with data using a small set of intuitive functions. Each function is a “verb”: the first argument is always the data, and the following arguments describe the change you want. The verbs never modify the original object, they return a new one, so nothing happens until you assign the result.

The way of tidyverse to store data is on `tibble` objects. A `tibble` is a dataframe with extra rules that makes the object more usable for modern standards. In our case, in practice, the term is interchangeable with dataframe. You can create a tibble with the function `as_tibble()`. Tibble objects in general print more clearly and tell you the type of each column.

```
sleep <- as_tibble(sleep)
sleep
```

```
## # A tibble: 634 x 14
```

```
## student_id study gender first_generation underrepresented avg_sleep_hours
## <int> <int> <chr> <int> <int> <dbl>
## 1 293 1 Female 0 0 6.17
## 2 294 1 Female 0 0 6.78
## 3 297 1 Female 0 1 4.23
## 4 303 1 Female 0 0 6.86
```

```
## 5      304      1 Female      0      1      6.8
## 6      305      1 Male      1      1      5.78
## 7      306      1 Male      0      0      5.67
## 8      308      1 Male      0      1      6.91
## 9      314      1 Male      0      0      6.35
## 10     315      1 Female    0      0      7.17
## # i 624 more rows
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

4 Data Transformation

4.1 Missing values, handled systematically

On Day 2 we met NA (a value that is unknown). On Day 4 we learned to find missing values. Today we handle them on purpose, before any analysis.

First, count them. `is.na()` tests each element, and `colSums()` adds up the TRUE values (R treats TRUE as 1 and FALSE as 0) column by column.

```
colSums(is.na(sleep))
```

```
##           student_id           study           gender
##                0                0                0
## first_generation underrepresented avg_sleep_hours
##                4                1                0
## daytime_sleep_minutes sleep_midpoint_minutes bedtime_variability
##                0                0                0
## nights_tracked_fraction prior_gpa term_gpa
##                0                0                0
##           term_units term_load_z
##           147           147
```

Two very different kinds of missingness show up here, and telling them apart matters. A handful of values are missing in `first_generation` and `underrepresented`: these are scattered, a few students who did not report something. But `term_units` and `term_load_z` are missing for 147 whole rows. That is not random: those columns were simply never collected for one of the five study samples. Structured missingness like this is common in real data, and how you treat it depends on why it is missing.

NB! When we have a missing value on a character vector as in: `gender`. The function `is.na()` will not count it. For R a blank in a character vector is not an NA.

```
sum(sleep$gender=="")
```

```
## [1] 3
```

The tidyverse tool for removing missing rows is `drop_na()`. On its own it removes every row that has a missing value in any column. With column names, it removes rows missing only in those columns.

```
# Removing rows missing ANYWHERE throws out all 147 rows and more (be careful)
nrow(sleep)
```

```
## [1] 634
```

```
nrow(drop_na(sleep))
```

```
## [1] 484
```

```
# Removing rows missing only in a specific column is usually what you want
sleep_drop<-drop_na(sleep,term_gpa)
nrow(sleep_drop)
```

```
## [1] 634
```

```
# The base-R logic you already know, written as a filter
sleep_baseR<-sleep[!is.na(sleep$term_gpa),]
nrow(sleep_baseR)
```

```
## [1] 634
```

The habit to build: always check how many rows you have before and after removing missing values. A bare `drop_na()` here would silently delete every student from one whole university, just because that university did not record course load. Deleting more than you intended is one of the most common silent mistakes in data analysis.

When we only want a summary to ignore missing values (rather than delete rows), we pass `na.rm = TRUE` inside the summary function, as we will see below.

4.2 Choosing columns: `select()`

The base R way to choose columns is `data[, "VariableName"]`. The tidyverse verb is `select()`. The first argument is the data, then the columns you want, in the order you write them.

```
# Keep a few columns
select(sleep, student_id, avg_sleep_hours, prior_gpa, term_gpa)
```

```
## # A tibble: 634 x 4
```

```
##   student_id avg_sleep_hours prior_gpa term_gpa
##   <int>         <dbl>      <dbl>   <dbl>
## 1      293           6.17        4     3.76
## 2      294           6.78        4     3.55
## 3      297           4.23        3     3.59
## 4      303           6.86        3.8    3.46
## 5      304           6.8         3.17   3.5
## 6      305           5.78        2.37   2.54
## 7      306           5.67        3.4     3.5
## 8      308           6.91        2.53   2.4
## 9      314           6.35        2.79   3.37
## 10     315           7.17        4     3.61
```

```
## # i 624 more rows
```

```
# Drop columns with a minus sign
select(sleep, -daytime_sleep_minutes, -sleep_midpoint_minutes)
```

```
## # A tibble: 634 x 12
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1      293     1 Female            0            0            6.17
## 2      294     1 Female            0            0            6.78
## 3      297     1 Female            0            1            4.23
## 4      303     1 Female            0            0            6.86
## 5      304     1 Female            0            1            6.8
## 6      305     1 Male              1            1            5.78
## 7      306     1 Male            0            0            5.67
## 8      308     1 Male            0            1            6.91
## 9      314     1 Male            0            0            6.35
## 10     315     1 Female            0            0            7.17
## # i 624 more rows
## # i 6 more variables: bedtime_variability <dbl>, nights_tracked_fraction <dbl>,
## #   prior_gpa <dbl>, term_gpa <dbl>, term_units <int>, term_load_z <dbl>
```

```
# Select helpers match columns by pattern
select(sleep, student_id, starts_with("term_"))
```

```
## # A tibble: 634 x 4
##   student_id term_gpa term_units term_load_z
##   <int>     <dbl>     <int>     <dbl>
## 1      293     3.76       49     0.112
## 2      294     3.55       49     0.112
## 3      297     3.59       46    -0.604
## 4      303     3.46       39    -2.28
## 5      304     3.5        39    -2.28
## 6      305     2.54       49     0.112
## 7      306     3.5        45    -0.843
## 8      308     2.4        48    -0.127
## 9      314     3.37       49     0.112
## 10     315     3.61       47    -0.366
## # i 624 more rows
```

Other useful helpers are `ends_with()` and `contains()`. To rename a column while keeping it, use `rename(new_name = old_name)`.

4.3 Choosing rows: `filter()` and `arrange()`

`filter()` keeps rows that meet one or more conditions. The comparison operators are the ones you know: `==` equal, `!=` not equal, `<` and `>` less or greater than, `<=` and `>=` less or greater than or equal.

```
# One condition: students who slept under six hours on average
filter(sleep, avg_sleep_hours < 6)
```

```
## # A tibble: 132 x 14
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1      297     1 "Female"            0            1            4.23
## 2      305     1 "Male"              1            1            5.78
```

```
## 3      306      1 "Male"      0      0      5.67
## 4      321      1 "Female"    0      0      5.96
## 5      334      1 ""          0      0      5.26
## 6      343      1 "Female"    0      0      5.51
## 7      360      1 "Male"      0      0      5.84
## 8      361      1 "Male"      0      0      5.96
## 9      378      1 "Female"    0      0      4.98
## 10     379      1 "Female"    0      0      5.72
```

```
## # i 122 more rows
```

```
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

```
# Several conditions separated by a comma means AND
filter(sleep, avg_sleep_hours < 6, prior_gpa >= 3.5)
```

```
## # A tibble: 67 x 14
```

```
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1      321      1 Female            0            0            5.96
## 2      361      1 Male              0            0            5.96
## 3      378      1 Female            0            0            4.98
## 4      390      1 Female            0            0            4.72
## 5      419      1 Male              0            0            5.67
## 6      430      1 Female            0            0            5.02
## 7      453      1 Male              0            0            4.72
## 8      483      1 Female            0            0            5.6
## 9      518      1 Female            1            0            4.95
## 10     525      1 Female            0            0            5.85
```

```
## # i 57 more rows
```

```
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

```
# The vertical bar | means OR
filter(sleep, avg_sleep_hours < 6 | avg_sleep_hours > 8)
```

```
## # A tibble: 161 x 14
```

```
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1      297      1 "Female"            0            1            4.23
## 2      305      1 "Male"              1            1            5.78
## 3      306      1 "Male"              0            0            5.67
## 4      321      1 "Female"            0            0            5.96
## 5      334      1 ""          0            0            5.26
## 6      343      1 "Female"            0            0            5.51
## 7      360      1 "Male"              0            0            5.84
## 8      361      1 "Male"              0            0            5.96
## 9      378      1 "Female"            0            0            4.98
```

```
## 10      379      1 "Female"          0          0          5.72
## # i 151 more rows
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

```
# Membership in a set with %in%
filter(sleep, study %in% c(1, 5))
```

```
## # A tibble: 208 x 14
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1      293     1 Female            0            0            6.17
## 2      294     1 Female            0            0            6.78
## 3      297     1 Female            0            1            4.23
## 4      303     1 Female            0            0            6.86
## 5      304     1 Female            0            1            6.8
## 6      305     1 Male              1            1            5.78
## 7      306     1 Male              0            0            5.67
## 8      308     1 Male              0            1            6.91
## 9      314     1 Male              0            0            6.35
## 10     315     1 Female            0            0            7.17
## # i 198 more rows
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

arrange() is a function that sorts the rows. Wrap a variable in desc() to sort from high to low.

```
arrange(sleep, avg_sleep_hours)
```

```
## # A tibble: 634 x 14
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1      427     3 Male            0            0            3.25
## 2      487     1 Female            0            1            3.66
## 3      615     1 Male              1            0            4.17
## 4      297     1 Female            0            1            4.23
## 5        55     2 Female            0            1            4.26
## 6    91551     4 Male              0            0            4.34
## 7      432     1 Male              0            0            4.39
## 8      278     5 Female            0            0            4.45
## 9    82986     4 Female            0            1            4.54
## 10      20     2 Female            0            0            4.57
## # i 624 more rows
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

```
arrange(sleep, desc(avg_sleep_hours))
```

```
## # A tibble: 634 x 14
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1         2     2 Female             1             0             9.79
## 2       180     5 Female             0             0             9.57
## 3       374     3 Male              0             0             8.89
## 4        14     2 Female             0             0             8.64
## 5       130     2 Female             0             0             8.53
## 6       517     3 Female             1             0             8.53
## 7        58     2 Female             0             0             8.52
## 8       560     3 Male              1             1             8.49
## 9       403     3 Male              0             0             8.48
## 10      496     3 Male              1             1             8.45
## # i 624 more rows
## # i 8 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>
```

4.4 Creating new variables: mutate()

`mutate()` adds a new column (or changes an existing one). You give the name of the new column and the formula for its values. Because the calculation is vectorised, it happens row by row automatically.

```
sleep<-mutate(sleep, gpa_change = term_gpa - prior_gpa)
sleep
```

```
## # A tibble: 634 x 15
##   student_id study gender first_generation underrepresented avg_sleep_hours
##   <int> <int> <chr>          <int>          <int>          <dbl>
## 1       293     1 Female             0             0             6.17
## 2       294     1 Female             0             0             6.78
## 3       297     1 Female             0             1             4.23
## 4       303     1 Female             0             0             6.86
## 5       304     1 Female             0             1             6.8
## 6       305     1 Male              1             1             5.78
## 7       306     1 Male              0             0             5.67
## 8       308     1 Male              0             1             6.91
## 9       314     1 Male              0             0             6.35
## 10      315     1 Female             0             0             7.17
## # i 624 more rows
## # i 9 more variables: daytime_sleep_minutes <dbl>,
## #   sleep_midpoint_minutes <dbl>, bedtime_variability <dbl>,
## #   nights_tracked_fraction <dbl>, prior_gpa <dbl>, term_gpa <dbl>,
## #   term_units <int>, term_load_z <dbl>, gpa_change <dbl>
```

```
select(sleep, student_id, prior_gpa, term_gpa, gpa_change)
```

```
## # A tibble: 634 x 4
##   student_id prior_gpa term_gpa gpa_change
##   <int>      <dbl>   <dbl>   <dbl>
## 1      293        4     3.76   -0.240
## 2      294        4     3.55   -0.45
## 3      297        3     3.59    0.59
## 4      303       3.8     3.46   -0.34
## 5      304       3.17     3.5    0.33
## 6      305       2.37     2.54    0.17
## 7      306       3.4     3.5    0.100
## 8      308       2.53     2.4   -0.130
## 9      314       2.79     3.37    0.58
## 10     315        4     3.61   -0.39
## # i 624 more rows
```

This is a classic transformation: “derive a variable from two existing columns”. A positive `gpa_change` means the student’s grades rose from the previous term, a negative value means they fell.

4.5 Recoding and categorising: `case_when()` and `recode()`

Often we want to turn a number into a category, or replace one set of labels with another.

`case_when()` reads conditions from top to bottom and returns the value of the first one that is TRUE. Order matters: once a row matches, later conditions are not checked.

```
sleep <- mutate(sleep,
  sleep_band = case_when(
    avg_sleep_hours < 6 ~ "under 6h",
    avg_sleep_hours < 7 ~ "6 to 7h",
    avg_sleep_hours >= 7 ~ "7h or more",
    TRUE ~ "unknown"
  )
)

count(sleep, sleep_band)
```

```
## # A tibble: 3 x 2
##   sleep_band    n
##   <chr>      <int>
## 1 6 to 7h      295
## 2 7h or more  207
## 3 under 6h   132
```

The last line, `TRUE ~ "unknown"`, is the catch-all. Without it, any row matching none of the conditions (for example a missing sleep value) would come out as NA. A forgotten catch-all is a quiet way to manufacture missing values.

`recode()` is simpler: it swaps specific values for new ones, written as `old = new`. It also has arguments for the leftovers, which is handy here because `first_generation` is coded 0 and 1 but contains one stray value and a few NAs.

```
sleep_recode<-mutate(sleep, first_gen = recode(as.character(first_generation),
      "0" = "Continuing generation",
      "1" = "First generation",
      .default = "Uncoded",
      .missing = "Missing"))

# How many observations each
table(sleep_recode$first_gen)
```

```
##
## Continuing generation      First generation      Missing
##           526              103              4
##           Uncoded
##           1
```

Use `recode()` for a fixed list of exact replacements, and `case_when()` when the new value depends on a condition rather than an exact match. The two-way cousin of `case_when()` is `if_else(condition, value_if_true, value_if_false)`, useful when there are only two outcomes.

4.6 Summarizing

`summarize()` (also `summarise()`) collapses many rows into a single summary value. It works with any function that takes a vector and returns one number. As the other tidyverse functions, `summarize` first argument is the dataset and the second is the name of the variable in the result. Remember `na.rm = TRUE` so that a missing value does not turn the whole result into NA.

```
summarize(sleep, mean_sleep = mean(avg_sleep_hours, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   mean_sleep
##   <dbl>
## 1      6.62
```

Across the whole sample students averaged about 6.6 hours a night, and roughly one in five slept under six hours.

5 Pipes

The tidyverse package also provides an approach for chaining a sequence of functions. This is the pipe operator (`|>` or in previous versions `%>%`). A shortcut to insert a pipe in the code is `Ctrl+Shift+M` (`Cmd+Shift+M` on a Mac).

The pipe works by taking the output of its left hand side and inserting it as the first argument to the function on its right hand side. This means that `some_function(data)` could be rewritten using a pipe as `data |> some_function()`. Other function arguments are left untouched. The pipe is what lets us read a whole analysis as a sentence: take the data, then filter, then group, then summarize.

5.1 group_by()

`group_by()` makes the summary run separately for each group. It changes nothing on its own, it only tags the data so the next verbs respect the groups. `n()` counts rows in a group

`andn_distinct()` counts unique values.

```
sleep |>
  group_by(sleep_band) |>
  summarize(
    mean_term_gpa = mean(term_gpa, na.rm = TRUE),
    n              = n()
  ) |>
  ungroup()
```

```
## # A tibble: 3 x 3
##   sleep_band mean_term_gpa     n
##   <chr>         <dbl> <int>
## 1 6 to 7h         3.49   295
## 2 7h or more     3.52   207
## 3 under 6h       3.25   132
```

And the headline comparison, using a simple two-way split:

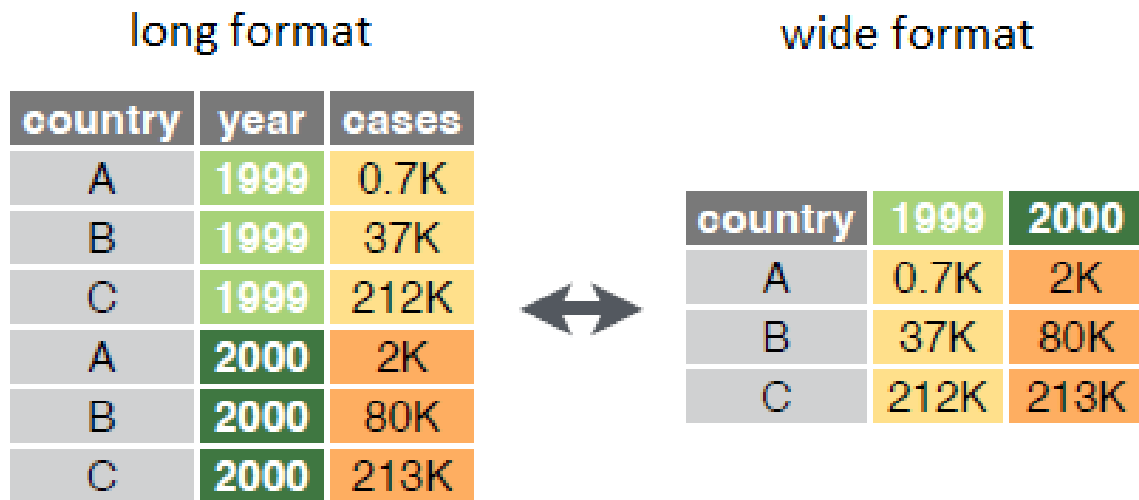
```
sleep |>
  mutate(under6 = if_else(avg_sleep_hours < 6, "under 6h", "6h or more")) |>
  group_by(under6) |>
  summarize(mean_term_gpa = mean(term_gpa, na.rm = TRUE))
```

```
## # A tibble: 2 x 2
##   under6      mean_term_gpa
##   <chr>         <dbl>
## 1 6h or more     3.50
## 2 under 6h      3.25
```

Students who averaged under six hours ended the term around 3.25, against about 3.50 for everyone else. *Always ungroup() when you save a grouped result*, otherwise later operations will silently keep grouping the data.

6 Reshaping data

Data can be stored in two shapes. In long format each measurement sits in its own row. In wide format those measurements spread across several columns. Different tasks need different shapes, so we move between them.



6.1 Converting to long

Here, `prior_gpa` and `term_gpa` are two measurements of the same thing (a GPA) at two different times, held in two columns. To compare across time, it helps to gather them into one column. `pivot_longer()` collapses several columns into two: one holding the old names, one holding the values.

```
gpa_long <- sleep |>
  select(student_id, study, prior_gpa, term_gpa) |>
  pivot_longer(cols = c(prior_gpa, term_gpa),
               names_to = "term", values_to = "gpa")
```

`gpa_long`

```
## # A tibble: 1,268 x 4
##   student_id study term      gpa
##   <int> <int> <chr> <dbl>
## 1      293     1 prior_gpa  4
## 2      293     1 term_gpa  3.76
## 3      294     1 prior_gpa  4
## 4      294     1 term_gpa  3.55
## 5      297     1 prior_gpa  3
## 6      297     1 term_gpa  3.59
## 7      303     1 prior_gpa  3.8
## 8      303     1 term_gpa  3.46
## 9      304     1 prior_gpa  3.17
## 10     304     1 term_gpa  3.5
## # i 1,258 more rows
```

Now each student has two rows, one per term, which makes a grouped summary easy.

```
gpa_long |>
  group_by(term) |>
  summarize(mean_gpa = mean(gpa, na.rm = TRUE))
```

```
## # A tibble: 2 x 2
##   term      mean_gpa
##   <chr>      <dbl>
## 1 prior_gpa    3.47
## 2 term_gpa     3.45
```

6.2 Converting to wide

`pivot_wider()` is the inverse: it spreads a names column and a values column back into several columns.

```
gpa_long |>
  pivot_wider(names_from = term, values_from = gpa)
```

```
## # A tibble: 634 x 4
##   student_id study prior_gpa term_gpa
##   <int> <int>    <dbl>    <dbl>
## 1     293     1      4      3.76
## 2     294     1      4      3.55
## 3     297     1      3      3.59
## 4     303     1     3.8     3.46
## 5     304     1     3.17    3.5
## 6     305     1     2.37    2.54
## 7     306     1     3.4     3.5
## 8     308     1     2.53    2.4
## 9     314     1     2.79    3.37
## 10    315     1      4     3.61
## # i 624 more rows
```

One thing to watch: after `pivot_longer()`, the new term column is text, not a number. That is fine here, but if names were years you would often convert them with `as.integer()`.

7 Joining two datasets

Real analysis usually means combining tables. Our main file records each student's study only as a number. The studies table turns that number into a university, a semester and a cohort size. We attach it with a join.

`left_join()` keeps every row of the left (first) table and adds the matching columns from the right (second) table. The `by` argument names the key column the two tables share.

```
nrow(sleep)
```

```
## [1] 634
```

```
sleep_joined <- sleep |> left_join(studies, by = "study")
nrow(sleep_joined)
```

```
## [1] 634
```

```
sleep_joined |>
  select(student_id, study, university, semester, cohort_n) |>
  head()
```

```
## # A tibble: 6 x 5
##   student_id study university semester cohort_n
##   <int> <int> <chr>      <chr>      <int>
## 1      293     1 Carnegie Mellon University Spring 2018     77
## 2      294     1 Carnegie Mellon University Spring 2018     77
## 3      297     1 Carnegie Mellon University Spring 2018     77
## 4      303     1 Carnegie Mellon University Spring 2018     77
## 5      304     1 Carnegie Mellon University Spring 2018     77
## 6      305     1 Carnegie Mellon University Spring 2018     77
```

The row count did not change. That is the point of `left_join()`: it never drops rows from the left table. Here the lookup has one row per study and the main data has many students per study, so this is a one-to-many join: each study's university is copied onto every matching student. Any left row with no match would simply receive NA in the new columns, rather than disappearing. This is precisely what the exam asks you to observe and explain.

A word of caution about keys. In this data `student_id` is only unique within a study: the same id number is reused across different samples. Joining on `student_id` would therefore match the wrong students together. To identify one student you need the pair (`study`, `student_id`), and to attach study information the correct key is `study`. Choosing the right key is half of getting a join right.

The join family, in short:

- `left_join()` keeps all rows of the left table
- `inner_join()` keeps only rows that match in both
- `full_join()` keeps all rows from both tables

8 Bringing everything together

Here is a single pipe that uses most of today's verbs. It derives a variable, joins the study information, filters to one group, keeps a tidy set of columns, and sorts the result.

```
cmu_short_sleepers <- sleep |>
  left_join(studies, by = "study") |>
  filter(university == "Carnegie Mellon University", avg_sleep_hours < 6) |>
  select(student_id, university, avg_sleep_hours,
         prior_gpa, term_gpa, gpa_change) |>
  arrange(gpa_change)
```

```
cmu_short_sleepers
```

```
## # A tibble: 53 x 6
##   student_id university avg_sleep_hours prior_gpa term_gpa gpa_change
##   <int> <chr>          <dbl>      <dbl>    <dbl>    <dbl>
## 1      356 Carnegie Mellon Uni~ 5.06      3.22    1.38    -1.84
## 2      299 Carnegie Mellon Uni~ 5.01      3.28    1.7     -1.58
## 3      453 Carnegie Mellon Uni~ 4.72      3.6     2.18    -1.42
```

```
## 4      487 Carnegie Mellon Uni~      3.66      2.78      1.5      -1.28
## 5      615 Carnegie Mellon Uni~      4.17      3.58      2.55      -1.03
## 6      211 Carnegie Mellon Uni~      4.87      3.77      2.78      -0.99
## 7      137 Carnegie Mellon Uni~      5.15      3.53      2.55      -0.98
## 8      525 Carnegie Mellon Uni~      5.85      3.55      2.6       -0.95
## 9      306 Carnegie Mellon Uni~      5.51      3.56      2.98      -0.58
## 10     321 Carnegie Mellon Uni~      5.96      4        3.44      -0.56
## # i 43 more rows
```

Read it as a sentence and you: derive a variable, join a second table, subset to a group, keep the only columns you need.

9 Tidyverse and R base programming

The tidyverse is written in R and relies on R for its infrastructure; there is no tidyverse without R. From a wider perspective, the tidyverse can be seen as sub-dialect of the R language.

The path to R begins with mastering a small number of relevant functions. Tidyverse provides the path of least resistance for data scientists interested in R. It enables a relatively inexperienced user to tidy up a messy data set quickly and easily.

There is no unique approach. Some data science tasks might be more easily performed using tidyverse or R base programming, others with a combination of the two.

Advantages

- Uniform standards for package organization.
- The idea of “verbs” to obtain tidy data
- The pipe operator (`|>`) channels the flow of tidy operations

Limitations

- Data are forced into a rectangular data structure.
- Tidyverse packages are designed for solving specific data science tasks; R packages allows a wider range of statistical analyses and modelling.

10 Glossary

Function	What it does
Import	
<code>read.csv()</code>	Reads a csv file into a data frame
Missing values	
<code>is.na()</code>	Tests each element for a missing value
<code>colSums(is.na())</code>	Counts missing values in every column
<code>drop_na()</code>	Removes rows with NA (all columns, or named ones)
<code>na.omit()</code>	Base R removal of rows with any NA
<code>na.rm = TRUE</code>	Tells a summary function to ignore NA
Columns	
<code>select()</code>	Keeps or drops columns
<code>starts_with(), contains()</code>	Select columns by a name pattern

Function	What it does
<code>rename()</code>	Renames a column
Rows	
<code>filter()</code>	Keeps rows that meet conditions
<code>arrange()</code>	Sorts rows (<code>desc()</code> for high to low)
<code>%in%</code>	Tests membership in a set of values
Creating and recoding variables	
<code>mutate()</code>	Adds or changes a column
<code>case_when()</code>	Builds a column from ordered conditions
<code>recode()</code>	Replaces specific values with new ones
<code>if_else()</code>	Chooses between two outcomes
Summarizing	
<code>summarize()</code>	Collapses rows into summary values
<code>group_by()</code>	Splits later operations by group
<code>ungroup()</code>	Removes the grouping
<code>n(), n_distinct()</code>	Count rows, count unique values
Reshaping	
<code>pivot_wider()</code>	Spreads one column into many
<code>pivot_longer()</code>	Gathers many columns into two
Joining	
<code>left_join()</code>	Keeps all left rows, adds matching right columns
<code>inner_join()</code>	Keeps only matched rows
<code>full_join()</code>	Keeps all rows from both tables
<code>by =</code>	Names the key column(s) to match on
Pipe	
<code> > (%>%)</code>	Passes the left result as the first argument on the right

11 Exercises

For these exercises you will use a different dataset from the one in class. Two files are provided. `happiness.csv` has one row per country (122 countries) with its world happiness score, its Human Development Index (HDI, stored as an integer scaled by 1000, so 928 means 0.928), its world region and hemisphere, and average alcohol consumption per person split into beer, spirits and wine. `regions.csv` is a small lookup that maps each world region to a continent and to the number of countries it contains.

1. Import and check. Import both files with `read.csv()` and show the first six rows of `happiness.csv`. How many countries and variables are there? Use `colSums(is.na())` to find which columns have missing values, then report how many rows remain after `drop_na()`.
2. Derive and subset. Create `total_alcohol` as beer plus spirits plus wine. Keep only the countries in Western Europe, and only the columns Country, HappinessScore and `total_alcohol`, sorted from the highest `total_alcohol` to the lowest.
3. Categorise. Use `case_when()` to label each country “low” (happiness under 5), “medium” (5 up to 6.5) or “high” (6.5 and above), with a catch-all. Count the countries in each band and find the mean `total_alcohol` per band.

4. Join. Attach the regions table with `left_join()` by “Region”. Compare the number of rows before and after. Then filter to the rows where continent is NA: which countries are they, and why did they not match?
5. Reshape. Select Country and the three alcohol columns, then use `pivot_longer()` to gather beer, spirits and wine into a drink type column and a servings column. Compute the mean servings for each drink type.